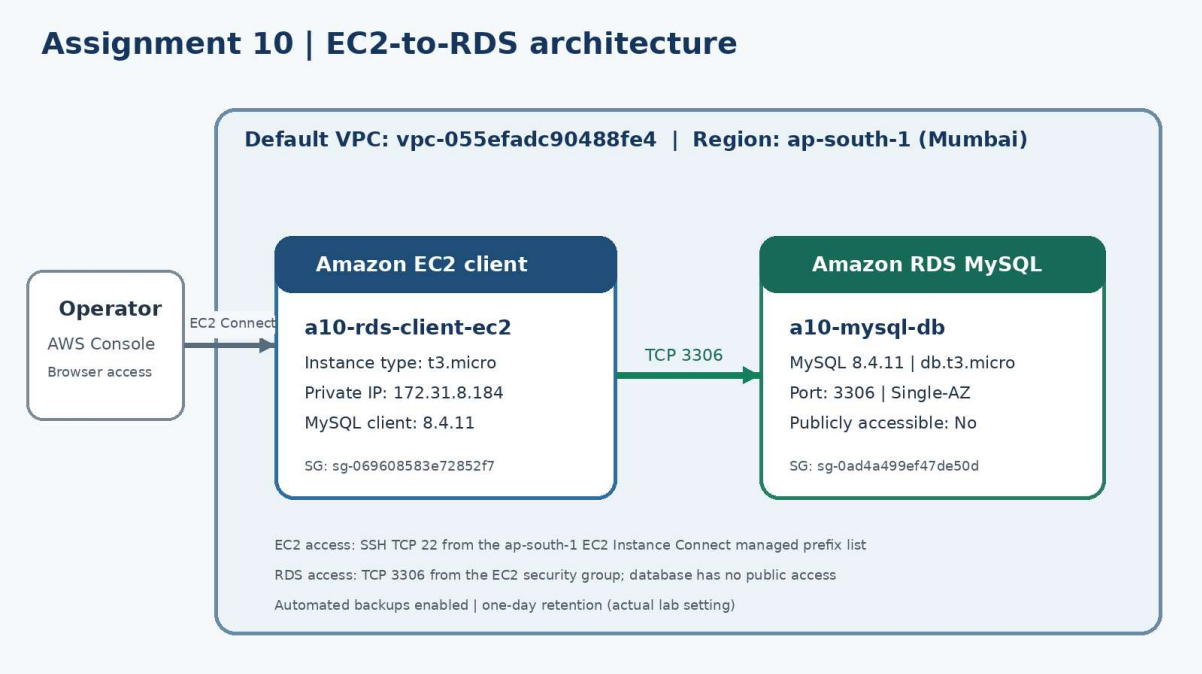
ASSIGNMENT 10

RDS Database Setup and Management

Amazon RDS for MySQL | Amazon EC2 | SQL Operations | Backups and Monitoring



Executive Summary

This practical demonstrates the setup and management of a private MySQL database using Amazon Relational Database Service (Amazon RDS), with an Amazon EC2 instance acting as the database client. The work covers network isolation, security-group rules, database provisioning, command-line connectivity, relational schema creation, SQL data operations, automated backups, and monitoring.

The RDS instance was created as a MySQL 8.4.11 DB instance in the Mumbai region. It uses a db.t3.micro instance class, 20 GiB General Purpose SSD (gp3) storage, encryption, and a private endpoint. An Ubuntu EC2 instance was connected to it through the VPC on MySQL TCP port 3306. The practical database company_db contains departments and employees tables, and the captured outputs show successful INSERT, SELECT, filtering, sorting, JOIN, UPDATE, DELETE, and aggregation operations.

> **Implementation note:** The deployed lab configuration is Single-AZ and uses one-day automated backup retention. The assignment mentions Multi-AZ for production and requests seven-day retention. These differences are disclosed in Section 8; the report does not claim those settings were implemented.

At a glance

Area	Recorded configuration / outcome
Region and VPC	Asia Pacific (Mumbai), ap-south-1; VPC vpc-055efadc90488fe4
EC2 client	a10-rds-client-ec2; t3.micro; private IPv4 172.31.8.184
RDS instance	a10-mysql-db; MySQL 8.4.11; db.t3.micro; Single-AZ
Network access	RDS publicly accessible: No; MySQL port 3306 from EC2 security group
Storage and encryption	20 GiB gp3; encryption enabled; storage autoscaling disabled
Backups	Automated backups enabled; one-day retention; an automated snapshot shown as Available
SQL exercise	2 tables; 3 departments; 5 initial employees; 4 employees after one update and one delete
Monitoring	CloudWatch/RDS graphs captured for CPU, connections, disk, I/O balance, memory, and storage

Contents

- 1. Objective and scope
- 2. Architecture and environment
- 3. Security groups and EC2 setup
- 4. Amazon RDS configuration
- 5. EC2-to-RDS connection
- 6. Database schema and SQL implementation
- 7. SQL queries and results
- 8. Automated backups and monitoring
- 9. Deviations, limitations, and recommendations
- 10. Conclusion and references

1. Objective and Scope

The objective was to provision and manage a MySQL database with Amazon RDS, restrict database access to an EC2 client, enable automated backups, connect to the database from EC2, create relational tables, execute sample SQL queries, and inspect RDS backup and monitoring information.

Tasks addressed

- Create EC2 and RDS security groups in the same VPC.
- Launch an Ubuntu EC2 instance and access it with EC2 Instance Connect.
- Create a MySQL 8.4.11 RDS instance with private connectivity.
- Enable automated backups and inspect the backup summary.
- Install the MySQL command-line client on EC2 and connect to RDS.
- Create the company_db database, departments table, and employees table.
- Insert sample data and execute retrieval, filtering, sorting, join, update, delete, and aggregate queries.
- Inspect database performance metrics in the RDS Monitoring view.

2. Architecture and Environment

The EC2 instance is used as a jump/client host inside the same VPC as RDS. The laptop reaches the EC2 terminal through the AWS EC2 Instance Connect service. The EC2 instance then connects to the RDS endpoint over the VPC on TCP port 3306. RDS is not publicly accessible, so the laptop does not connect directly to the database endpoint.

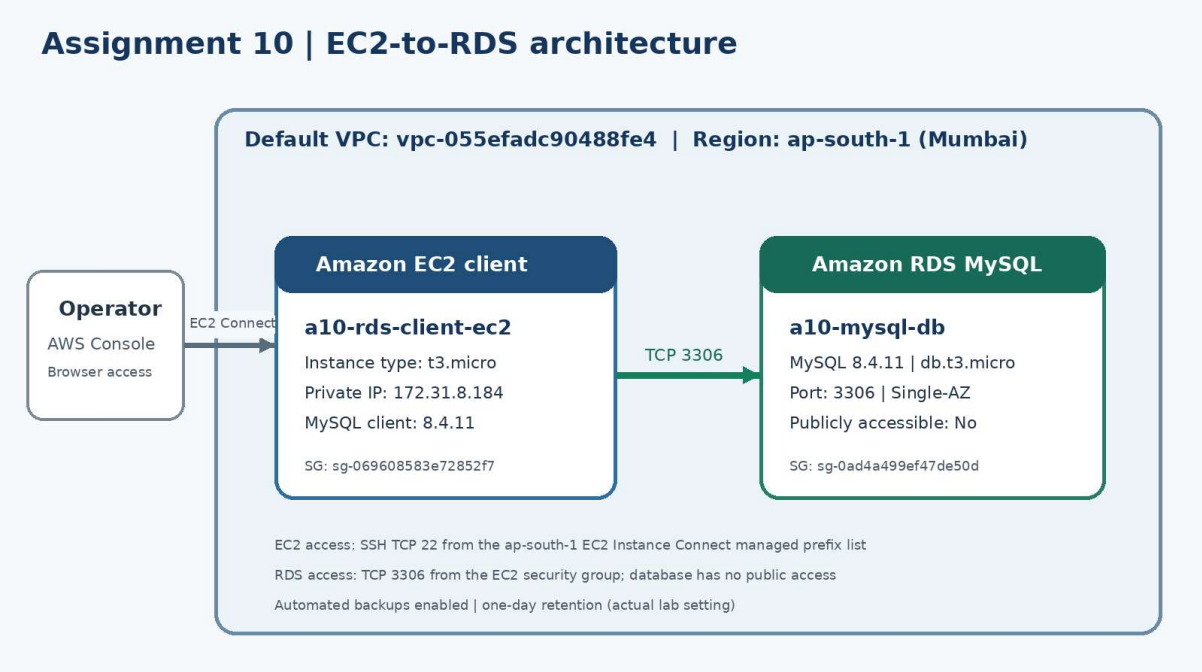


Figure 1. Logical architecture implemented for the practical.

Environment summary

Component	Observed value
AWS Region	Asia Pacific (Mumbai), ap-south-1
VPC	vpc-055efadc90488fe4 (Default VPC)
EC2 instance ID	i-003b761e72acc262e
EC2 instance name / type	a10-rds-client-ec2 / t3.micro
EC2 Availability Zone	ap-south-1b
EC2 private IPv4	172.31.8.184
RDS DB identifier	a10-mysql-db
RDS endpoint	a10-mysql-db.cpgmaqa8a2d6.ap-south-1.rds.amazonaws.com
RDS port / AZ	3306 / ap-south-1b
RDS public accessibility	No

3. Security Groups and EC2 Setup

Two purpose-specific security groups were created in the same VPC. The EC2 group supports terminal access through the regional EC2 Instance Connect service. The RDS group permits MySQL traffic from the EC2 client group rather than opening the database to the internet.

Security group	ID	Inbound rule	Intended source
A10-EC2-SSH-SG	sg-069608583e72852f7	SSH / TCP 22	AWS-managed prefix list: com.amazonaws.ap-south-1.ec2-instance-connect (pl-0fa83cebf909345ca)
A10-RDS-MySQL-SG	sg-0ad4a499ef47de50d	MySQL/Aurora / TCP 3306	EC2 security group: sg-069608583e72852f7

EC2 client instance

The EC2 client instance was launched as a t3.micro in Availability Zone ap-south-1b. Its private address was 172.31.8.184. The instance reached the Ubuntu shell through browser-based EC2 Instance Connect; whoami returned ubuntu and hostname returned ip-172-31-8-184. The 2/2 EC2 status checks were passed in the instance screenshot.

MySQL client installation

```
sudo apt update
sudo apt install mysql-client -y
mysql --version
```

The terminal output confirms MySQL client 8.4.11-0ubuntu0.26.04.1 for Linux x86_64. The client is installed on the EC2 instance, not on the RDS host; RDS remains a managed database service.

Security interpretation

- RDS Publicly accessible was set to No.
- The EC2 and RDS resources used the same VPC, vpc-055efadc90488fe4. The RDS inbound rule allowed TCP port 3306 from the EC2 security group, not from a public CIDR range.

4. Amazon RDS MySQL Configuration

The DB instance was provisioned using the Standard create workflow. The configured initial database name was a10db. The account/console presented an error when the requested seven-day backup retention value was selected; one day was used for the working lab, and the resulting value is shown in the submitted evidence.

Setting	Observed configuration
DB identifier	a10-mysql-db
Engine / version	MySQL Community, 8.4.11
DB instance class	db.t3.micro (2 vCPU, 1 GiB RAM shown)
Deployment	Single-AZ; Multi-AZ = No
Master username	dbadmin (password is not reproduced in this report)
Initial database	a10db
VPC / subnet group	vpc-055efadc90488fe4 / default-vpc-055efadc90488fe4
Availability Zone	ap-south-1b
Public accessibility	No
Port	3306
Storage	20 GiB General Purpose SSD (gp3)
Provisioned IOPS / throughput	3,000 IOPS / 125 MiB/s as displayed
Storage autoscaling	Disabled
Encryption	Enabled; AWS KMS key alias aws/rds
Monitoring	Database Insights - Standard; Enhanced Monitoring disabled

RDS endpoint

```
a10-mysql-db.cpgmaqa8a2d6.ap-south-1.rds.amazonaws.com
Port: 3306
```

Initial database versus exercise database

RDS created the initial database a10db during provisioning. For the SQL exercise, a separate database named company_db was created from the MySQL session and used for the departments and employees tables. Both databases therefore appear in SHOW DATABASES.

5. Connecting EC2 to RDS

After EC2 Instance Connect opened the Ubuntu terminal, the MySQL command-line client was installed and used to connect to the private RDS endpoint. The connection used the master username dbadmin and prompted for the password interactively. No password is included in this report.

```
mysql --ssl-mode=REQUIRED \
  -h a10-mysql-db.cpgmaqa8a2d6.ap-south-1.rds.amazonaws.com \
  -P 3306 \
  -u dbadmin \
  -p
```

The session reached the MySQL monitor. The following verification commands succeeded:

```
SELECT VERSION();
SHOW DATABASES;
USE a10db;
SELECT DATABASE();
```

Verification	Observed output / meaning
SELECT VERSION()	8.4.11 - the RDS MySQL server responded.
SHOW DATABASES	Listed a10db and the standard MySQL system databases.
USE a10db	Database changed - the session selected the initial RDS database.
SELECT DATABASE()	Returned a10db - confirms the active database.

> **Result:** EC2-to-RDS connectivity was verified successfully over the private VPC connection using MySQL TCP port 3306.

6. Relational Database and SQL Implementation

A sample company employee-management database was built to demonstrate relational tables, primary keys, a foreign key, and practical SQL operations.

6.1 Database schema

Table	Column	Definition / purpose
departments	department_id	INT PRIMARY KEY; unique department identifier
departments	department_name	VARCHAR(50), NOT NULL, UNIQUE; department name
employees	employee_id	INT PRIMARY KEY; unique employee identifier
employees	first_name, last_name	VARCHAR(50), NOT NULL
employees	email	VARCHAR(100), UNIQUE
employees	salary	DECIMAL(10,2)
employees	hire_date	DATE
employees	department_id	INT foreign key referencing departments.department_id

> **Relationship:** departments.department_id is the parent primary key. employees.department_id is a foreign key, so each employee department reference must match an existing department.

6.2 Table creation SQL

```
CREATE TABLE departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    salary DECIMAL(10,2),
    hire_date DATE,
    department_id INT,
    FOREIGN KEY (department_id)
        REFERENCES departments(department_id)
);
```

6.3 Sample data inserted

Department ID	Department name
1	Engineering
2	Human Resources
3	Finance

Employee ID	Name	Department	Salary	Hire date
101	Aarav Shah	Engineering	65000.00	2024-01-15
102	Diya Patel	Engineering	58000.00	2023-11-20
103	Kabir Mehta	Human Resources	52000.00	2024-03-10
104	Anaya Desai	Finance	61000.00	2022-08-01
105	Ishaan Kulkarni	Human Resources	47000.00	2025-02-12

The insert statements reported 3 department rows and 5 employee rows affected, with no duplicate records and no warnings. The salary values are recorded as sample numeric values; no currency unit was specified in the schema.

7. SQL Queries and Results

The captured terminal outputs show the following operations. Query results are described at the point they were executed; the filtering, sorting, and JOIN queries ran before the subsequent UPDATE and DELETE operations.

7.1 Retrieve records

```
SELECT * FROM departments;
SELECT * FROM employees;
```

The first SELECT returned three departments and the second returned the five inserted employees.

7.2 Filter and sort

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary > 55000;

SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC;
```

The WHERE query returned Aarav Shah (65,000), Diya Patel (58,000), and Anaya Desai (61,000). The ORDER BY query displayed all five employees in descending salary order.

7.3 Join related tables

```
SELECT e.employee_id, e.first_name, e.last_name,
       d.department_name, e.salary
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id;
```

The INNER JOIN returned five rows and attached each employee to the corresponding department name.

7.4 Update an employee record

```
UPDATE employees
SET salary = 68000
WHERE employee_id = 101;

SELECT * FROM employees
WHERE employee_id = 101;
```

One row was matched and changed. Aarav Shah (employee_id 101) salary changed from 65,000.00 to 68,000.00.

7.5 Delete a record

```
DELETE FROM employees
WHERE employee_id = 105;

SELECT * FROM employees;
```

One row was deleted. Employee 105 (Ishaan Kulkarni) no longer appeared in the subsequent employee listing, which contained four rows.

7.6 Aggregate employees by department

```
SELECT d.department_name,
       COUNT(e.employee_id) AS total_employees
FROM departments d
LEFT JOIN employees e
  ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

Department	Final employee count
Engineering	2
Finance	1
Human Resources	1
Total	4

SQL result: All shown statements completed successfully. The final employee table contains four records after the update and deletion; all three departments remain in the database.

8. Automated Backups and Monitoring

8.1 Automated backups

Automated backups were enabled for a10-mysql-db. The submitted backup configuration screenshot shows a one-day retention period and one automated snapshot with Available status. The automated backup summary shows backup status active, encryption enabled, and a point-in-time recovery window.

Backup item	Observed evidence
Automated backups	Enabled
Retention period	1 day (actual setting)
Backup window	22:55-23:25 UTC
Snapshot type / status	Automated / Available
Earliest restorable time	November 9, 2026, 15:38 (UTC+05:30), as displayed
Latest restorable time	November 9, 2026, 16:10 (UTC+05:30), as displayed
Backup encryption	Enabled

Automated backups provide a recovery mechanism and point-in-time restore capability within the configured retention period. A restore operation was not performed as part of the captured practical; the evidence verifies backup availability and restore-time metadata, not a completed recovery test.

8.2 RDS monitoring

The RDS Monitoring tab was captured after database activity. The screenshot contains CloudWatch metric graphs for CPU utilization, database connections, volume IOPS/throughput exceeded checks, disk queue depth, EBS byte and I/O balance, freeable memory, and free storage space.

Metric	Observation from the captured graph
CPUUtilization	Activity was recorded; the plotted value was close to the lower graph range toward the end of the displayed period.
DatabaseConnections	The graph reached approximately one connection during the captured interval.
DataVolumeIOPSExceededCheck	No exceeded condition is shown in the displayed interval.
DataVolumeThroughputExceededCheck	No exceeded condition is shown in the displayed interval.
DiskQueueDepth	The graph dropped close to zero toward the end of the interval.
EBSByteBalance / EBSIOBalance	Both displayed balance graphs were approximately 99% near the latest point.
FreeableMemory	The graph showed memory availability in the approximate 53 MB range at the latest point.
FreeStorageSpace	The graph showed approximately 19.5 GiB of free space from the 20 GiB allocated volume.

9. Deviations, Limitations, and Recommendations

9.1 One-day backup retention instead of seven days

The assignment specifies a seven-day automated backup retention period. During this practical, selecting seven days in the AWS console produced an error in the account configuration, so the database was created and verified with one-day retention. The submitted screenshots confirm that the deployed value is one day. Therefore, the backup feature is implemented and verified, but the seven-day retention requirement is not met.

> **Reporting accuracy:** This report records the actual one-day value. It does not describe the seven-day requirement as completed and does not assume that every AWS Free Tier account universally has the same retention restriction.

9.2 Single-AZ deployment

The RDS instance configuration shows Multi-AZ = No. Single-AZ was used for this cost-conscious lab. For production workloads with higher availability requirements, use an appropriate Multi-AZ deployment and evaluate its additional instance/storage costs and failover behavior.

9.3 Evidence and testing boundaries

- The uploaded security-group screenshots show rule protocol and port but the horizontal view cuts off the Source column. The configured source selections are documented in this report; recapturing the full inbound-rule table with Source visible would strengthen the evidence.
- An automated snapshot was shown as Available and restore-time metadata was displayed. A database restore was not executed or verified.
- The Monitoring screenshot reflects a brief observation period and does not demonstrate sustained load, configured alarms, or production performance.
- The database contains demonstration employee data only. No application workload or production data migration was tested.

9.4 Cost and cleanup recommendations

The evidence set shows the EC2 instance and RDS database in use at capture time. The uploaded screenshots do not prove that these resources were subsequently deleted. After the report and evidence are safely stored, review and remove lab resources that are no longer required: delete the RDS DB instance without a final snapshot if no recovery copy is needed, inspect and remove retained automated backups/manual snapshots as appropriate, terminate the EC2 instance, verify its EBS volume was deleted, and remove the two custom security groups after dependencies are gone. Also review the Billing console for other billable resources. Deleting the DB instance alone does not automatically remove every separately retained snapshot or volume.

10. Conclusion

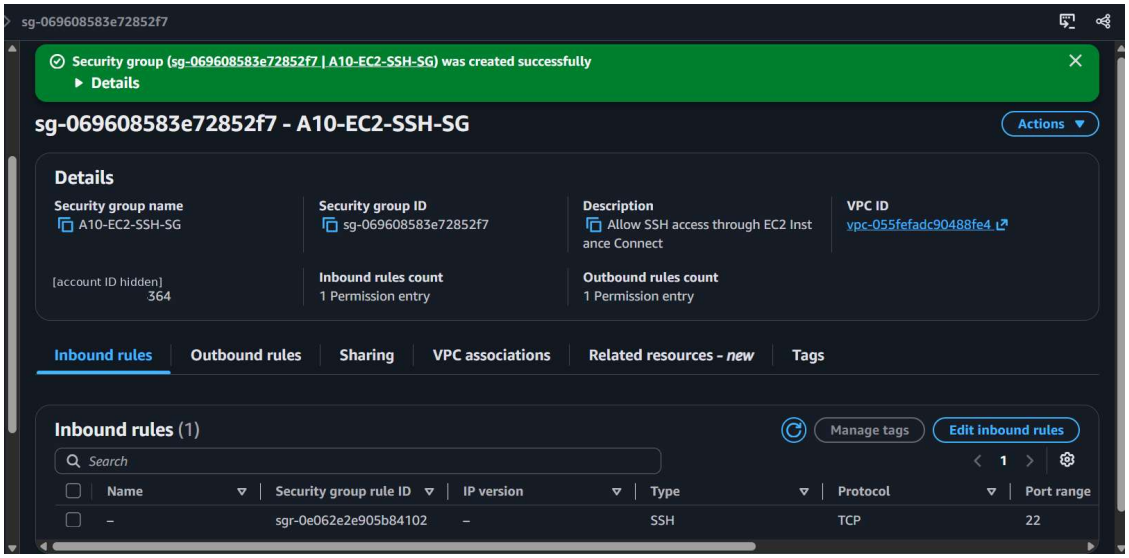
The practical successfully demonstrated provisioning of an Amazon RDS for MySQL database, private EC2-to-RDS connectivity, security-group-based access control, installation of a MySQL client, relational database/table creation, and common SQL data operations. Automated backups were enabled and an available automated snapshot was observed. RDS monitoring metrics were also reviewed.

The final database exercise produced three departments and five initial employees, then updated one employee salary and deleted one employee, leaving four employee records. The main implementation qualifications are that the RDS deployment was Single-AZ and the backup retention was one day rather than the requested seven days. These limitations are explicitly recorded so that the report matches the actual evidence.

Appendix A. Screenshot Evidence

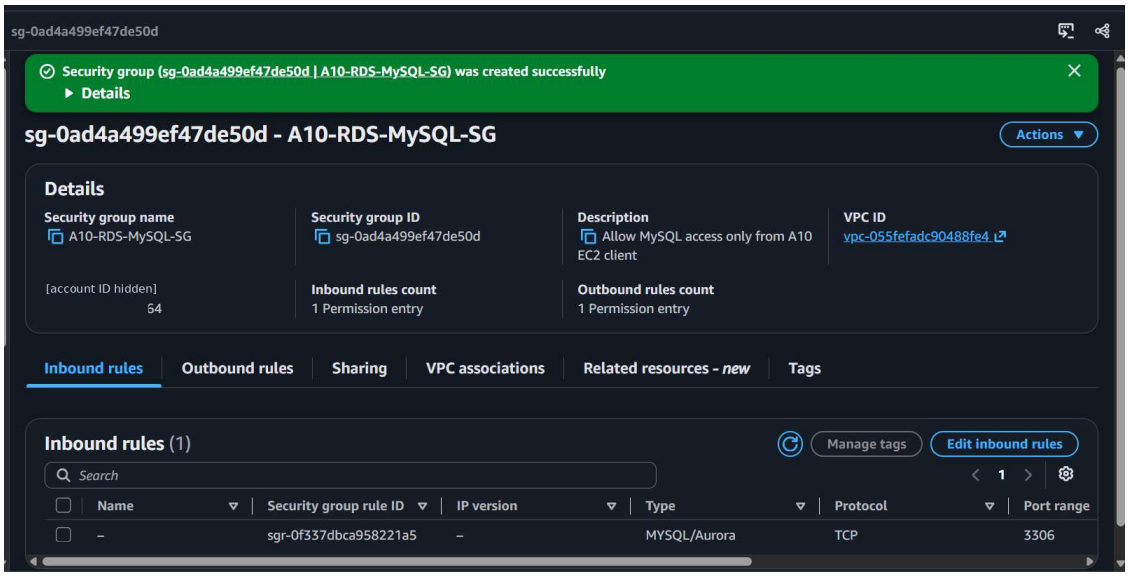
The following appendix embeds all 18 uploaded screenshots captured on 9 November 2026. Account identifiers visible in a small number of AWS console headers/owner fields have been masked in the report copy; the original screenshots remain unchanged in the uploaded ZIP. Figure captions describe the evidence visible in each screenshot.

Figure A01. EC2 SSH security group



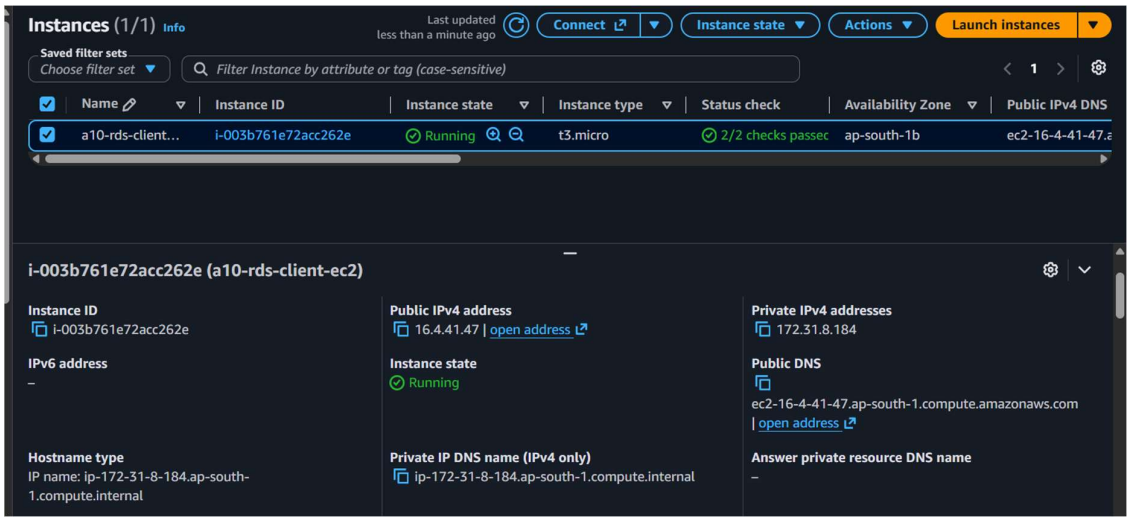
A dedicated EC2 security group was created in the Mumbai VPC. The visible rule is SSH over TCP port 22; the configured source was the AWS-managed EC2 Instance Connect prefix list for ap-south-1.

Figure A02. RDS MySQL security group



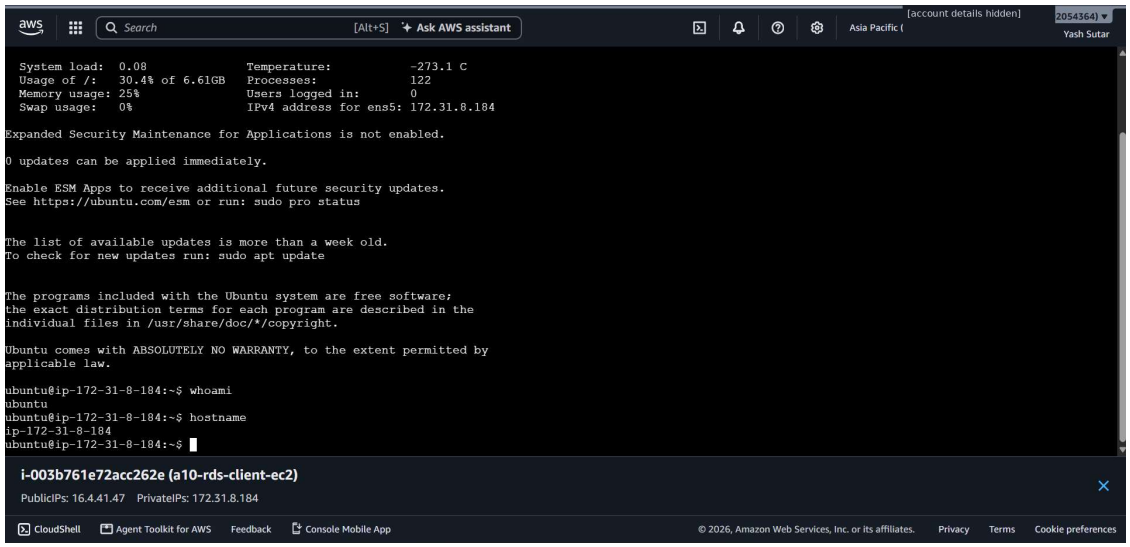
A separate security group was created for the database. The inbound rule is MySQL/Aurora over TCP port 3306, configured to accept traffic from the EC2 client security group. The source column is outside the captured horizontal viewport.

Figure A03. EC2 instance launched and healthy



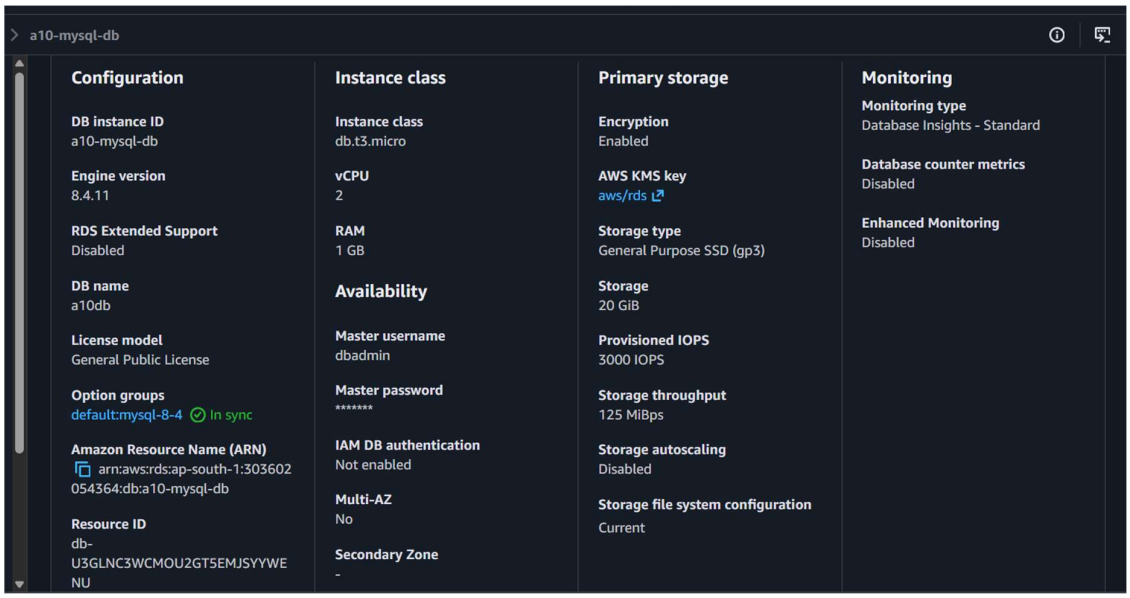
The client instance is running with instance type t3.micro, instance ID i-003b761e72acc262e, Availability Zone ap-south-1b, and private IPv4 address 172.31.8.184. The console shows 2/2 status checks passed.

Figure A04. Browser-based EC2 Instance Connect



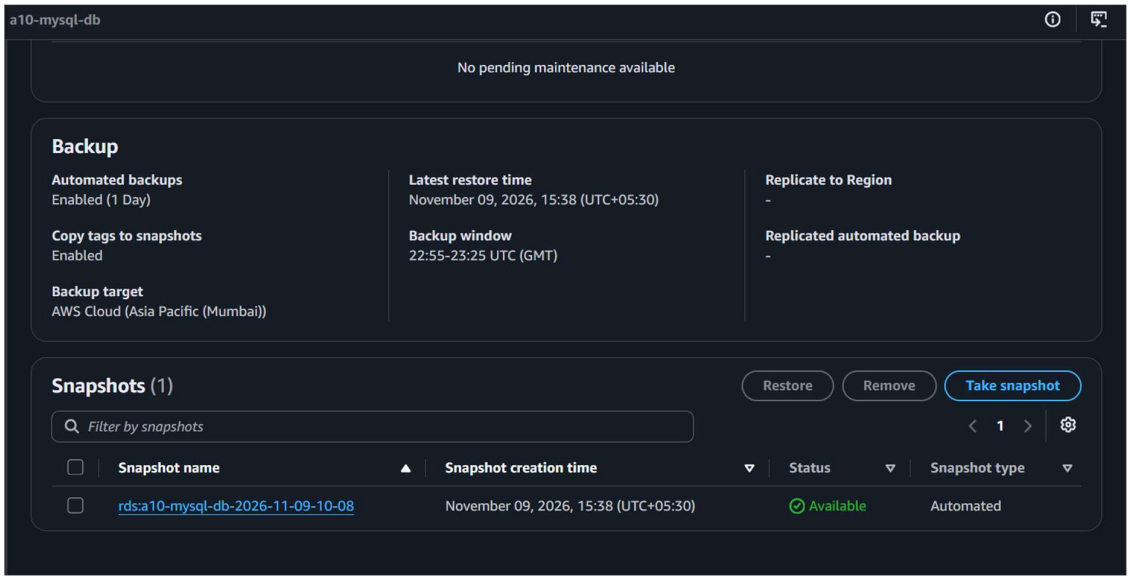
The Ubuntu shell was reached through EC2 Instance Connect. The whoami command returned ubuntu and hostname returned ip-172-31-8-184.

Figure A05. RDS MySQL instance configuration



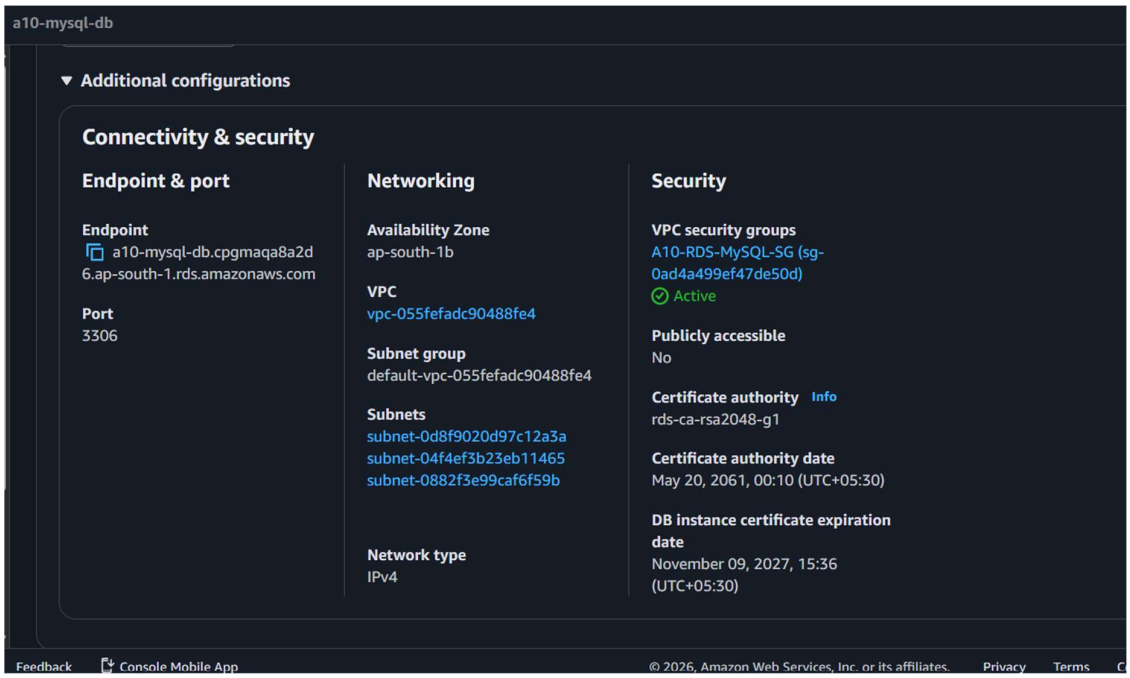
The RDS instance a10-mysql-db uses MySQL 8.4.11 and db.t3.micro. It is Single-AZ (Multi-AZ: No), with 20 GiB General Purpose SSD gp3 storage, encryption enabled, storage autoscaling disabled, and Database Insights Standard.

Figure A06. Automated backup settings



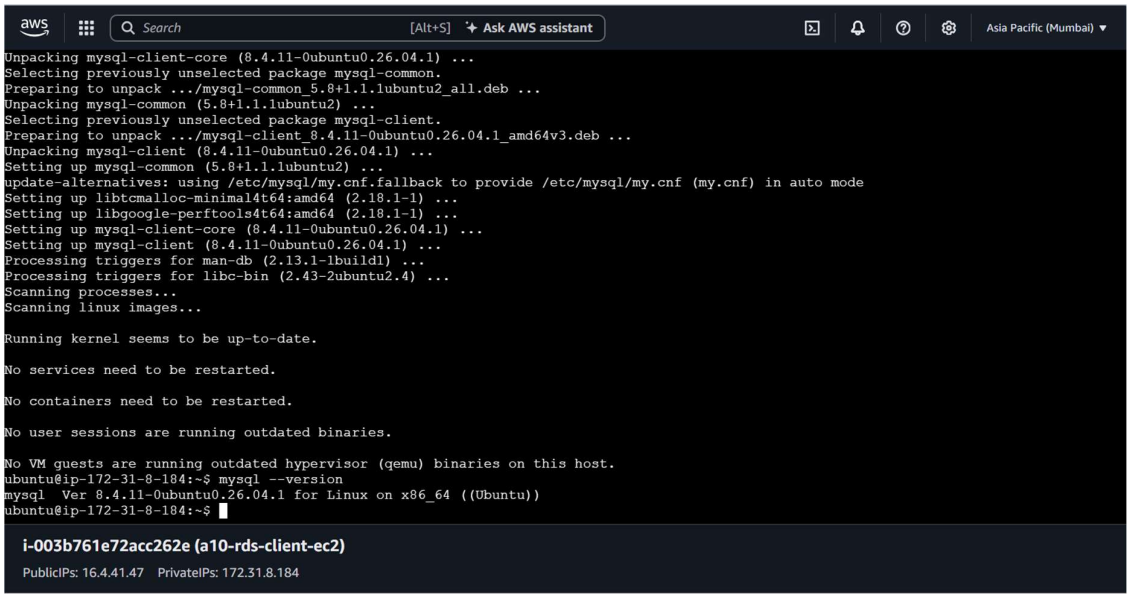
Automated backups are enabled with a one-day retention period. The console shows one automated snapshot in Available status. The backup window shown is 22:55-23:25 UTC.

Figure A07. RDS endpoint and private networking



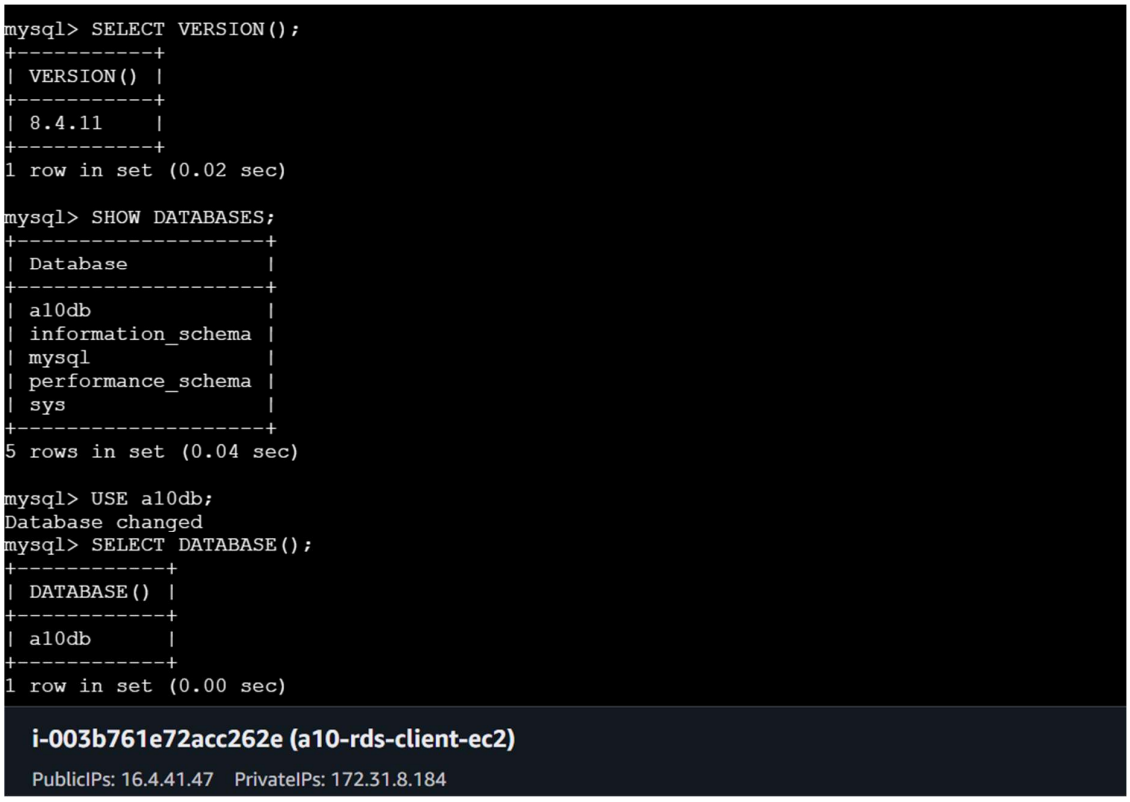
The database endpoint is a10-mysql-db.cpgmaqa8a2d6.ap-south-1.rds.amazonaws.com on port 3306. It is in VPC vpc-055efadc90488fe4 and publicly accessible is No.

Figure A08. MySQL client installed on EC2



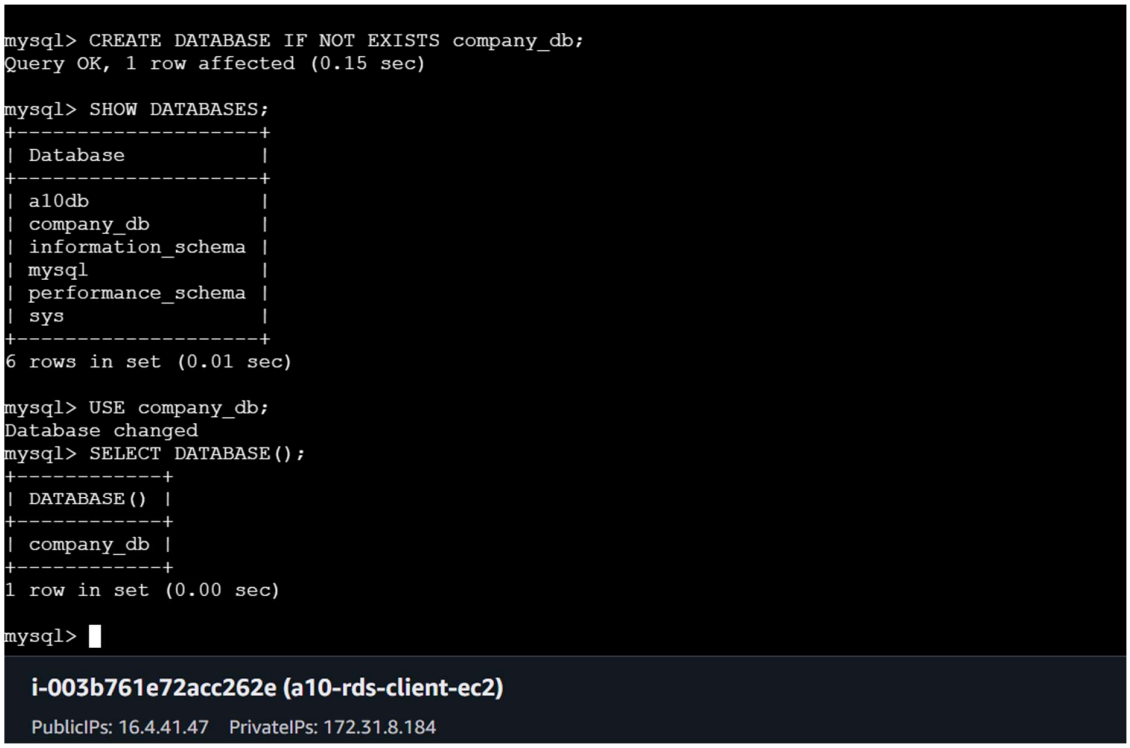
The Ubuntu EC2 terminal shows successful installation of mysql-client and a MySQL client version of 8.4.11-0ubuntu0.26.04.1.

Figure A09. EC2-to-RDS SQL connectivity verification



SQL commands executed from the EC2 terminal returned MySQL server version 8.4.11, listed the databases, selected a10db, and confirmed the active database with SELECT DATABASE().

Figure A10. Create and select company_db



The company_db database was created, listed by SHOW DATABASES, selected with USE company_db, and verified with SELECT DATABASE().

Figure A11. Create relational tables

```
mysql> CREATE TABLE departments (
    ->     department_id INT PRIMARY KEY,
    ->     department_name VARCHAR(50) NOT NULL UNIQUE
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE employees (
    ->     employee_id INT PRIMARY KEY,
    ->     first_name VARCHAR(50) NOT NULL,
    ->     last_name VARCHAR(50) NOT NULL,
    ->     email VARCHAR(100) UNIQUE,
    ->     salary DECIMAL(10,2),
    ->     hire_date DATE,
    ->     department_id INT,
    ->     FOREIGN KEY (department_id)
    ->     REFERENCES departments(department_id)
    -> );
Query OK, 0 rows affected (0.05 sec)

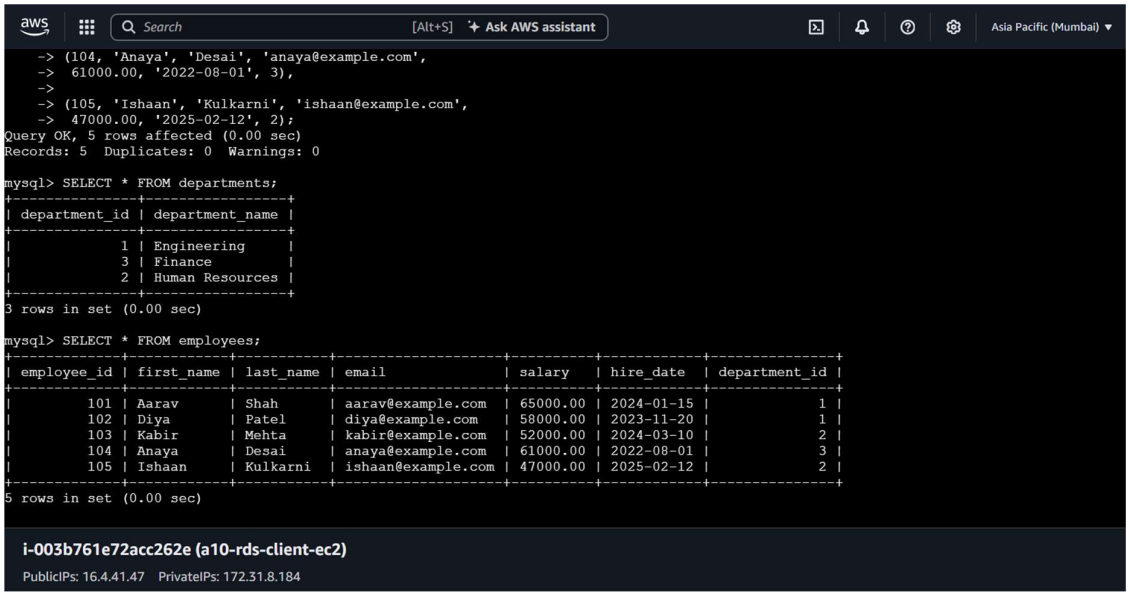
mysql> SHOW TABLES;
+----------------------+
| Tables_in_company_db |
+----------------------+
| departments          |
| employees            |
+----------------------+
2 rows in set (0.00 sec)
```

i-003b761e72acc262e (a10-rds-client-ec2)

PublicIPs: 16.4.41.47 PrivateIPs: 172.31.8.184

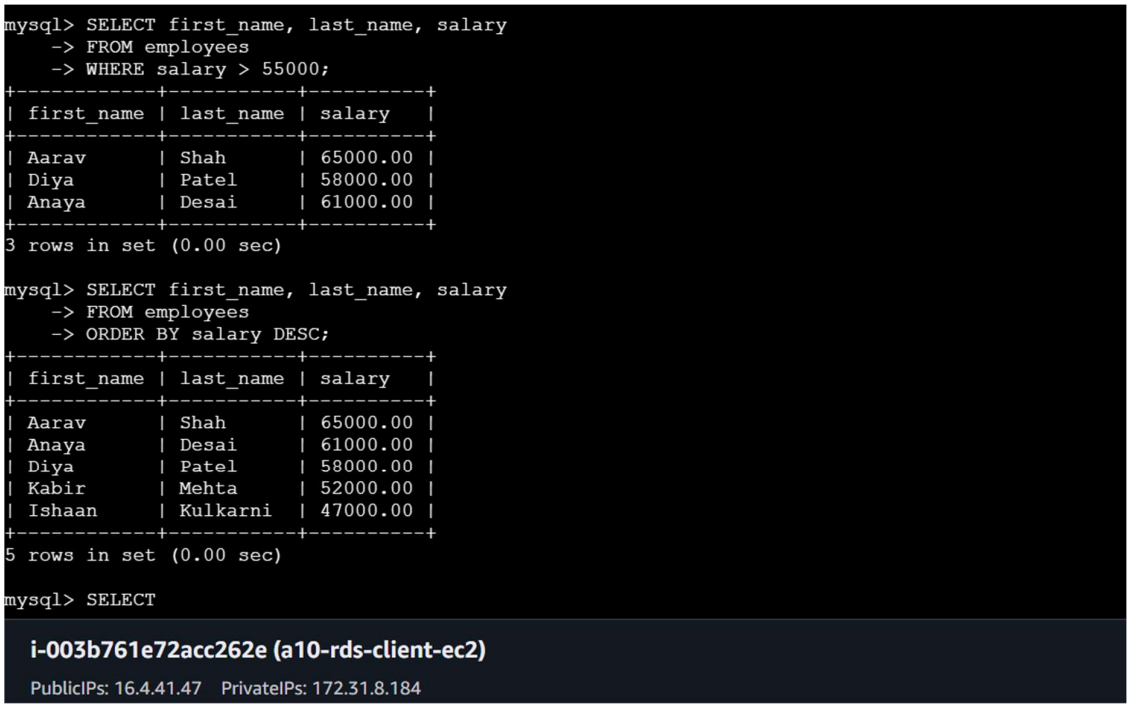
The departments and employees tables were created successfully. SHOW TABLES lists both tables.

Figure A12. Insert and verify sample records



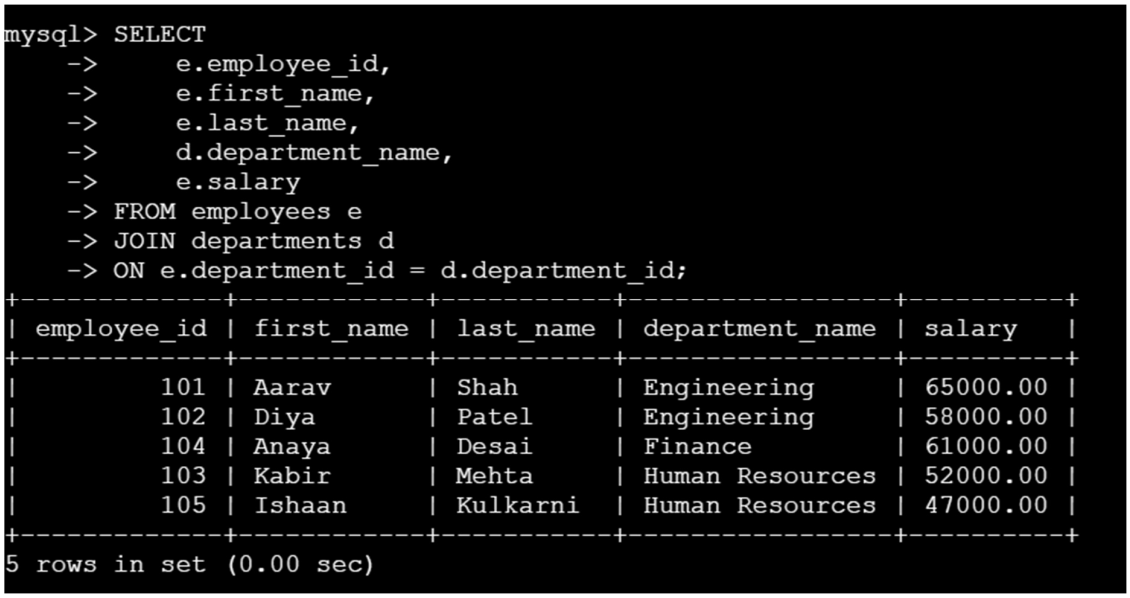
Three departments and five employee records were inserted. SELECT queries verified the rows in both tables.

Figure A13. SELECT, WHERE, and ORDER BY queries



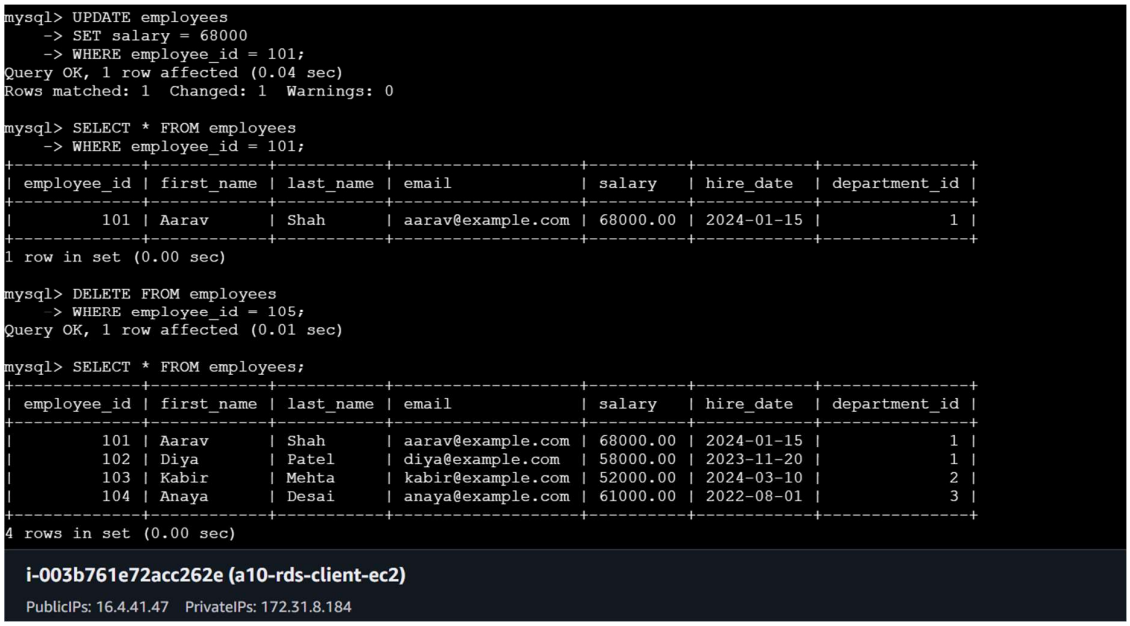
The salary filter returned three employees with salary greater than 55,000. The sort query listed all five original employees by salary in descending order.

Figure A14. INNER JOIN across related tables



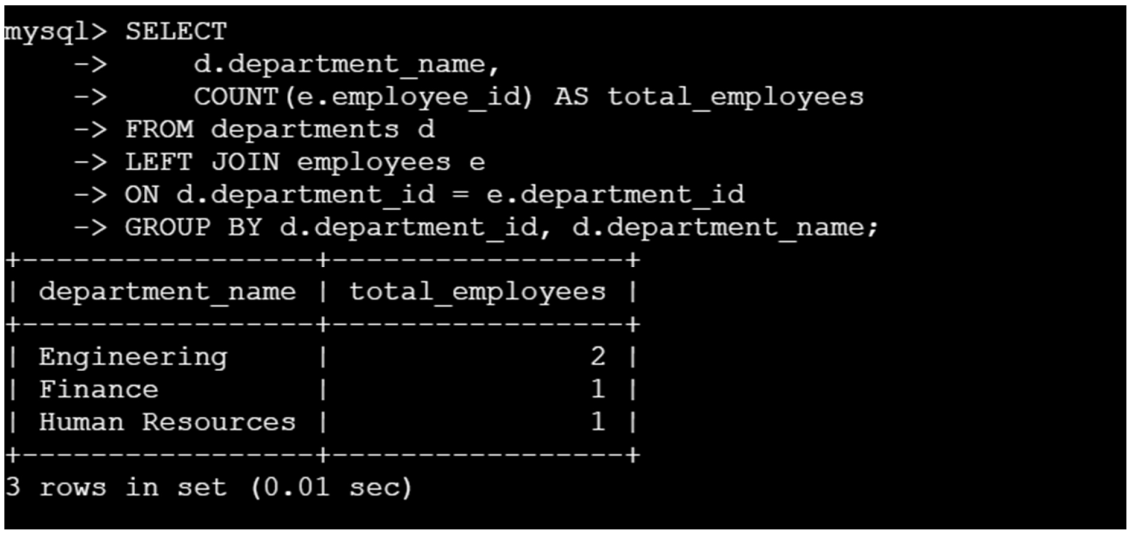
An INNER JOIN combined employee records with department names and returned five rows.

Figure A15. UPDATE and DELETE operations



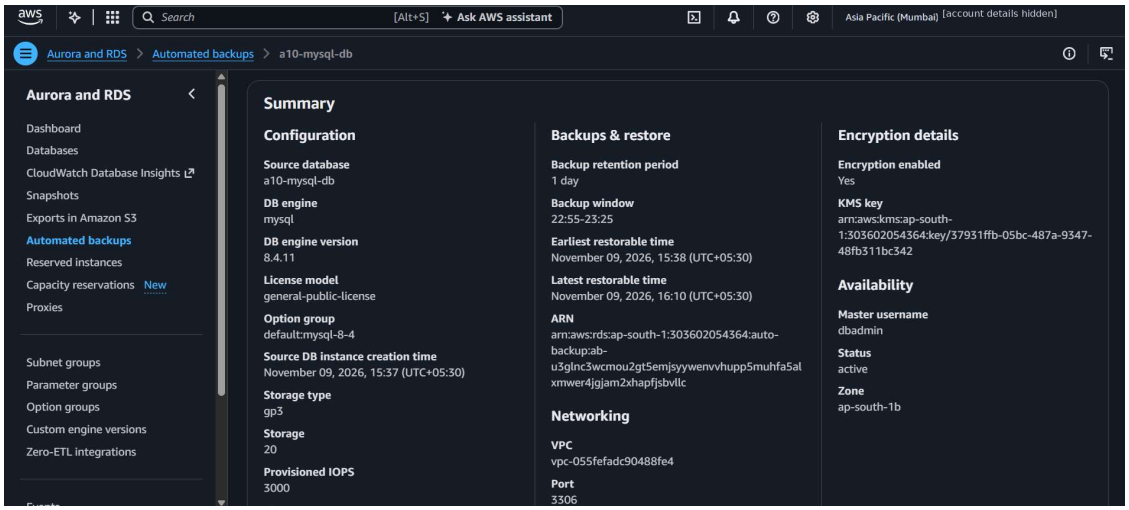
Employee 101 salary was changed from 65,000 to 68,000. Employee 105 was deleted. A subsequent SELECT showed four remaining employee rows.

Figure A16. Department-wise employee count



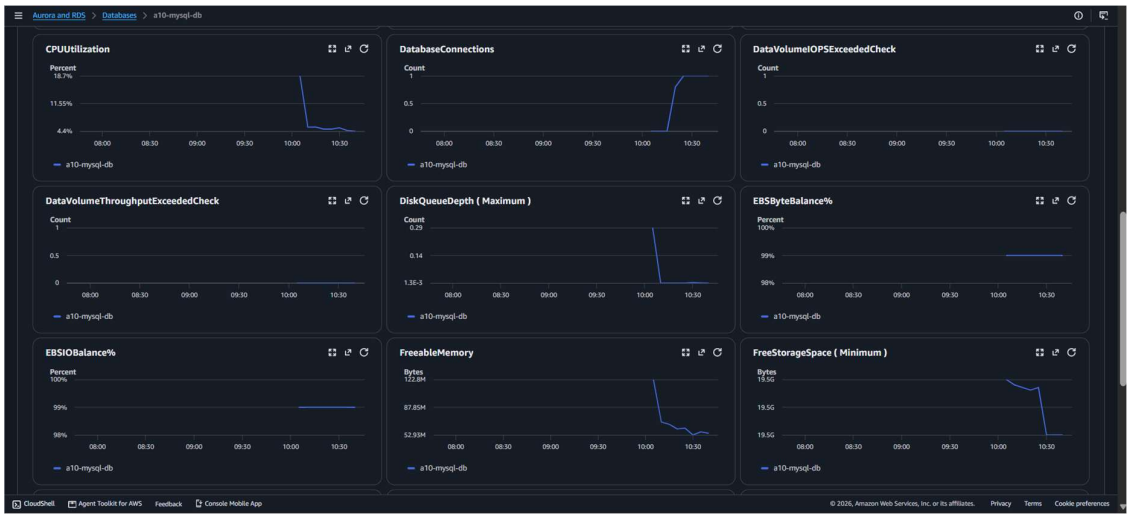
A LEFT JOIN with COUNT and GROUP BY returned Engineering: 2, Finance: 1, and Human Resources: 1.

Figure A17. Automated backup summary and restore window



The automated backup summary confirms one-day retention, active status, encryption enabled, and displayed earliest/latest restorable times on November 9, 2026.

Figure A18. RDS CloudWatch monitoring metrics



The monitoring view displays CPU utilization, database connections, disk queue depth, EBS balance metrics, freeable memory, free storage, and volume IOPS/throughput exceeded checks.